About this PDF

Built only from: **NOC:Natural Language Processing (NPTEL course book, weeks 2, 3)** — https://nptel.ac.in/courses/106105158

Licence: CC BY-SA 4.0. Accessed 2026-09-26.

Covers 7 of 8 syllabus sub-topics of this unit.

Partly covered in the source:

- Finite-State Automata
- Transducers for lexicon and rules, Tokenization

Not in the source (not covered in this PDF):

- Origins and challenges of NLP

Language Modeling: Grammar-based LM, Statistical LM

Definition

Grammar-based language models rely on explicit rule-based patterns such as “a determiner is followed by an adjective or noun,” encoding linguistic constraints directly into the model. Statistical language models learn probabilities of word sequences from a tagged training corpus, estimating the most likely part-of-speech tag or word given its context.

Explanation

In a rule-based approach, one examines the local context (e.g., the preceding word) and applies handcrafted if-then-else rules that capture permissible symbol patterns, such as prohibiting consecutive determiners. Statistical models use a corpus where each word is annotated with its true part-of-speech tag; from this data they derive probabilistic estimates of tag sequences or word sequences, allowing the model to predict the most probable tag for a new word based on learned statistics.

Example

A rule-based solution might state: “If the previous word is a determiner, the next word is highly likely to be an adjective or a noun,” which can be used to disambiguate part-of-speech tagging.

Key points

- Rule-based (grammar) models encode linguistic constraints like determiner-adjective ordering.
- Statistical models are trained on a tagged corpus to learn the most probable tag sequences.
- Transformation-based learning (TBL) derives rules from the training data to adjust initial tag predictions.
- Both approaches aim to use context information to improve tagging or word prediction.

Regular Expressions

Explanation

Finite state automata are directed graphs whose nodes (states) are connected by edges labeled with symbols. Starting from a designated start state, the automaton consumes input symbols one by one, following the corresponding labeled edges; if it reaches an accepting (final) state the input string is accepted. Such automata recognize regular languages, which are precisely the languages described by regular expressions, and any regular expression can be converted into an equivalent finite automaton.

Example

Consider an automaton that recognizes the words **color** and **colour**. Starting at the start state, the input “c” moves to state 1, “o” to state 2, and so on, eventually reaching an accepting state after processing the full word. The path exists for both “color” and “colour”, so both are accepted; the input “colr” fails because there is no transition from the intermediate state for the symbol “r”.

Key points

- A finite state automaton is a directed graph with states and labeled edges.
- It accepts a string if, after reading all symbols, it ends in an accepting (final) state.
- Regular languages are exactly those that can be described by regular expressions and recognized by such automata.

Finite-State Automata

Partly covered in the source.

Definition

A finite-state automaton (FSA) is a directed graph whose nodes are called **states** and whose edges are labeled with symbols (which may be empty). One state is designated as the **start state** and one or more states are marked as **accepting (final) states**. An FSA recognizes a regular language, i.e., the set of strings described by a regular expression.

Explanation

When an input string is processed, the automaton begins at the start state, reads the first character, and follows the edge labeled with that character to the next state. This procedure repeats for each subsequent character. If the traversal ends in an accepting state, the string is accepted; otherwise it is rejected. This mechanism allows the FSA to decide membership in a regular language in time linear in the length of the input string.

Example

The automaton shown in the lecture accepts the words **color** and **colour**: both strings can be followed through a sequence of edges that ends in the accepting state 6. The string **colr** cannot be accepted because after reaching state 3 there is no outgoing edge labeled ‘r’, so the traversal stops before an accepting state.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Algorithm / Steps

1. Begin at the designated start state.
2. Read the next input character.
3. Follow the edge whose label matches the character to move to the next state.
4. Repeat steps 2–3 until all characters are consumed.
5. If the current state is an accepting (final) state, accept the string; otherwise reject it.

Key points

- An FSA is a directed graph with states and labeled edges, including start and accepting states.
- It recognizes regular languages, which can be derived from regular expressions.
- Processing an input string proceeds character by character, moving along matching edges, and runs in linear time with respect to the string length.
- Both deterministic and nondeterministic FSAs can be used, as any NFA can be converted to an equivalent DFA.

English Morphology

Definition

Morphological analysis is the process of determining the lemma or root form of a word, such as finding that the lemma of *boys* is *boy*; it goes beyond simply recognizing whether a string belongs to a language.

Explanation

Finite-state automata (FSAs) can recognize whether a word is singular or plural by accepting strings that belong to a predefined regular language, but they cannot provide the underlying lemma. To obtain the root form, transducers—machines similar to FSAs but capable of producing output—are required for morphological analysis. Two-level morphology introduces an intermediate representation between the lexical form (e.g., *cat+N+PL*) and the surface form (e.g., *cats*). This intermediate form contains placeholders that may be null or a phoneme, allowing spelling-change rules to be applied in a separate step to generate the final surface form. The approach separates the handling of morpheme concatenation from spelling changes, simplifying the modeling of regular and irregular patterns.

Example

The word *unhappy* is generated by an automaton that first adds the prefix *un* to the adjective root *happy* and then may attach suffixes such as *-er*, *-est*, or *-ly* to produce *unhappier*, *unhappiest*, *unhappily*, etc. In a two-level system, the plural of *cat* is formed by creating an intermediate form *cat+ s* and then applying a rule that converts the placeholder to the surface form *cats*; similarly, *fox* becomes *foxes* via an intermediate representation that handles the spelling change.

Key points

- FSAs recognize word forms but cannot output lemmas; transducers are needed for full morphological analysis.
- Two-level morphology uses an intermediate form with placeholders to separate morpheme concatenation from spelling-change rules.
- Engineering approaches embed spelling-change handling within the finite-state model, avoiding separate rule processing.
- Morphological analysis can be modeled with finite-state methods, employing tools such as AT & FSM library or OPENFST.

Transducers for lexicon and rules, Tokenization

Partly covered in the source.

Definition

A transducer is like a finite-state automaton whose edges carry both an input symbol and an output symbol, allowing one string to be converted into another (e.g., “boys” → “boy”). In morphological processing, a two-level morphology uses an intermediate form and surface-level form, with rules that specify how characters change depending on their context.

Explanation

In a transducer each transition reads a character from the input string and writes a corresponding character to the output string, thus performing the required spelling changes. Context-sensitive rules describe these changes: a character *a* is replaced by *b* when it is preceded by *c* and followed by *d* (i.e., “a” between “c” and “d” becomes “b”). Such rules are applied to the intermediate representation to produce the final surface form.

Example

- The noun “cat” is regular; applying the plural rule yields “cats”.
- The noun “fox” is irregular; the plural form is produced as “foxes” by inserting an “e” before the “s”.
- A rule example: “a” → “b” when preceded by “c” and followed by “d”.

Key points

- A transducer maps input characters to output characters on each edge, enabling string-to-string conversion.
- Two-level morphology separates an intermediate lexical form from the surface form, using context-sensitive replacement rules.
- Context-sensitive rules depend on the surrounding characters of the target character.
- Regular and irregular morphological patterns can be handled by defining separate transducer paths or rules.

Detecting and Correcting Spelling Errors

Definition

Spelling correction is the task of detecting and fixing spelling errors that appear in noisy textual data before further analysis.

Explanation

The correction process relies on a distance metric that quantifies how similar two words are. Edit distance is commonly used as this metric because it measures the minimum number of operations needed to transform one word into another, allowing the system to identify words that are “very close” to the misspelled form.

Example

In the sentence “I am writing on behalf of,” the word *behalf* may appear as the misspelled token **behaf**, missing the final ‘l’. The correction aims to replace **behaf** with the correct word *behalf*.

Algorithm / Steps

1. Take the misspelled word as input.
2. Compute the edit distance between this word and each candidate word in the vocabulary.
3. Select the word(s) with the smallest edit distance as the closest match for correction.

Key points

- Spelling correction addresses noisy text by fixing incorrect words.
- Edit distance serves as a simple and effective similarity metric.
- The word with the smallest edit distance to the misspelled token is chosen as the correction.

Minimum Edit Distance

Definition

Minimum edit distance is the smallest number of edit operations required to transform one string into another. The edit operations considered are insertion, deletion, and substitution of characters. Formally, D(i,j) denotes the edit distance between the first *i* characters of string X and the first *j* characters of string Y; the overall distance between X and Y is D(n,m) where *n* and *m* are their lengths.

Explanation

- Deleting the i-th character of X gives a cost of D(i-1,j) + 1.
- Inserting the j-th character of Y gives a cost of D(i,j-1) + 1.
- Substituting (or matching) the i-th character of X with the j-th character of Y gives a cost of D(i-1,j-1) plus 0 if the characters are the same or a substitution cost otherwise.

By filling a matrix of size (n + 1) × (m + 1) with these recurrence relations, the entry D(n,m) yields the minimum edit distance.

AcademicsGen — source: NPTEL — CC BY-SA 4.0

Example

Transforming **intention** into **execution** requires five operations: one deletion, one insertion, and three substitutions. If each operation costs 1, the edit distance is 5 (Levenshtein distance). Using a variant where substitution costs 2, the distance becomes 8.

Algorithm / Steps

1. Initialise a matrix D of size (n + 1) × (m + 1).
2. Set D(0,0) = 0; D(i,0) = i (deleting i characters) and D(0,j) = j (inserting j characters).
3. For each i from 1 to n and each j from 1 to m compute:
   - costDelete = D(i-1,j) + 1
   - costInsert = D(i,j-1) + 1
   - costSubst = D(i-1,j-1) + c, where c = 0 if X[i] = Y[j] else substitution cost (1 or 2).
4. Set D(i,j) to the minimum of costDelete, costInsert, and costSubst.
5. After filling the matrix, D(n,m) is the minimum edit distance.

Key points

- Edit distance counts the minimum insertions, deletions, and substitutions needed to convert one string to another.
- It is computed efficiently with dynamic programming by building a matrix of sub-problem solutions.
- Different cost schemes (e.g., substitution = 1 or = 2) lead to different distance values such as 5 or 8 for the same pair of strings.
- The algorithm runs in O(n × m) time, filling each matrix cell with a constant-time minimum operation.

Source and licence

This PDF is adapted from **NOC:Natural Language Processing (NPTEL course book, weeks 2, 3)** (https://nptel.ac.in/courses/106105158), accessed 2026-09-26.

Source licence: **CC BY-SA 4.0** — https://creativecommons.org/licenses/by-sa/4.0/

Changes made by AcademicsGen: the source text was rephrased, reorganised and grammar-corrected into study material; nothing added.

This PDF is shared under the same licence (CC BY-SA 4.0).